

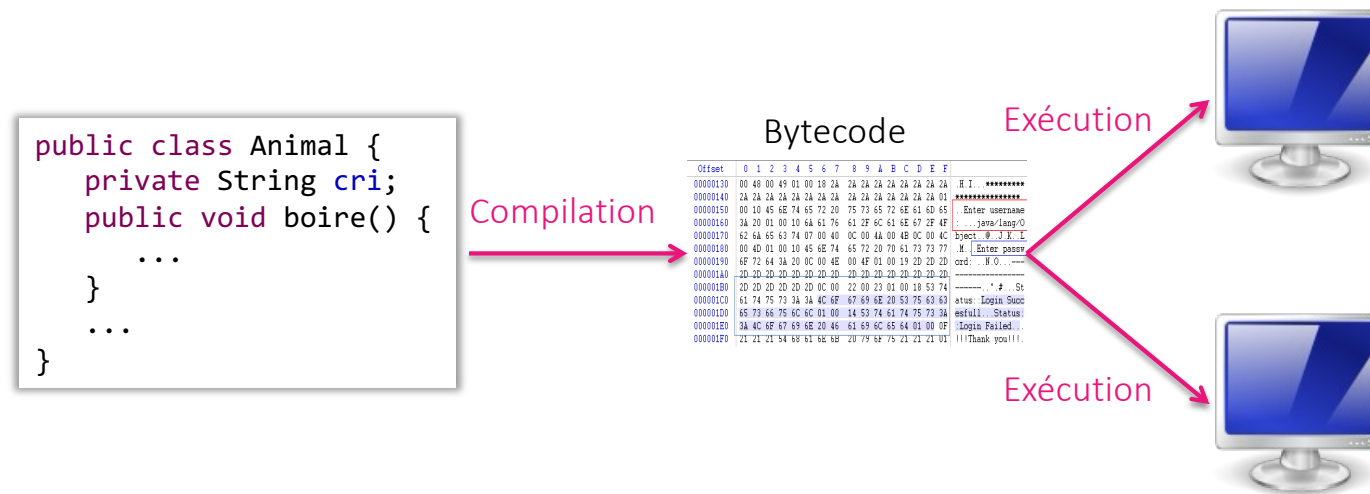
Processus de développement logiciel Java et automatisation avec Apache Ant

Stéphanie CHOLLET

Enseignant-Chercheur Grenoble INP – Esisar/LCIS

Objectifs : du code à l'exécution

Comment passer du code Java à l'exécution d'une application Java ?



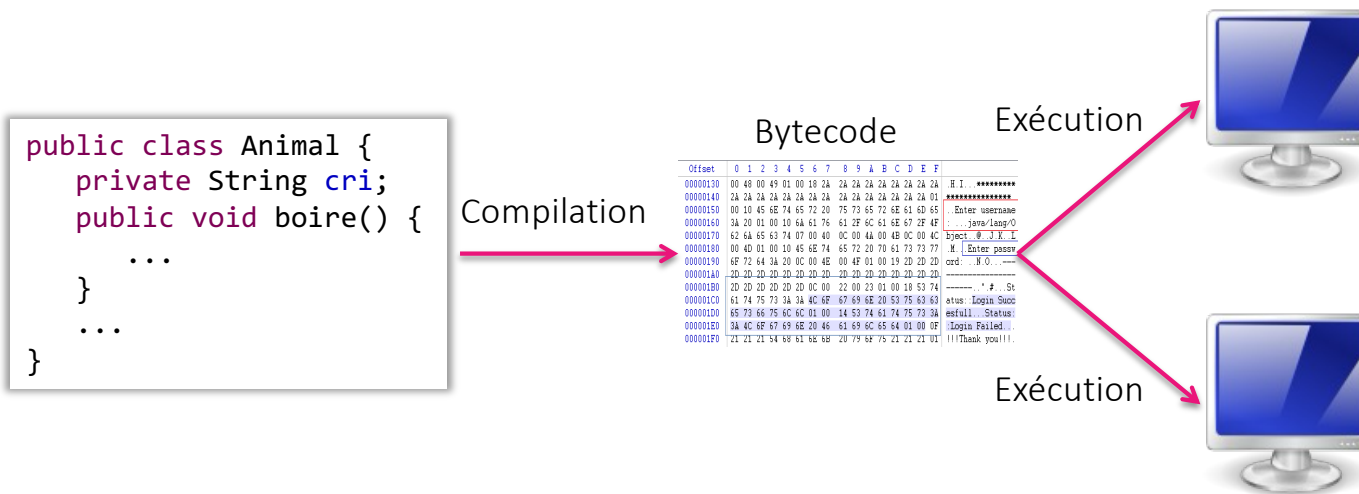
En lignes de commandes et de manière automatisée

Outils du processus de développement logiciel Java

En ligne de commandes

Rappel : du code à l'exécution

- ▶ Il faut des outils pour :
 - ▶ **Compiler** le code Java en bytecode
 - ▶ **Exécuter** le bytecode sur la machine hôte



Pour compiler

► Prérequis :

- Compilateur **javac** disponible dans le JDK (*Java Development Toolkit*)

► Commande :

<code>javac MonFichier.java</code>	Compile un fichier source Java
<code>javac *.java</code>	Compile tous les fichiers .java du dossier
<code>-d <répertoire></code>	Spécifie le dossier de sortie des fichiers .class (bytecode)
<code>-classpath <chemin></code> <code>-cp <chemin></code>	Définit le chemin des dépendances

► Exemple :

- `javac Zoo.java Animal.java Tortue.java Guepard.java`

Principe facile mais ce n'est pas la réalité !

Pour exécuter

- ▶ Prérequis :

- ▶ Commande **java** disponible dans un environnement d'exécution Java (*Java Runtime Environment*)

- ▶ Commande :

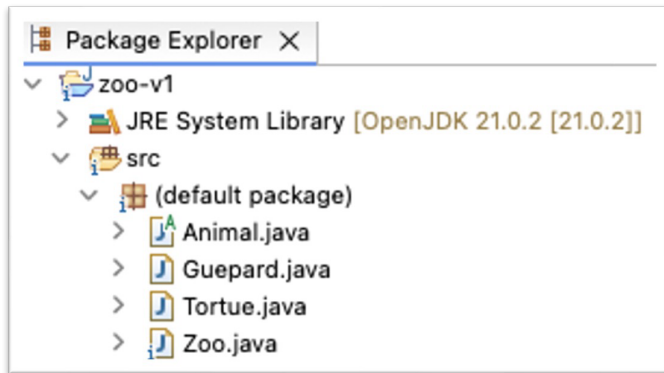
<code>java ClassePrincipale</code>	Exécute la ClassePrincipale Le nom de la classe principale doit être son nom pleinement qualifié
<code>-classpath <chemin></code> <code>-cp <chemin></code>	Définit le chemin vers les classes/dépendances (séparés par : sous Linux ou ; sous Windows)

- ▶ Exemple :

- ▶ `java Zoo`

Principe toujours facile mais ce n'est pas la réalité !

Limite de la compilation



```
src$ ls
Animal.java Guepard.java Tortue.java Zoo.java
src$ javac *.java
src$ ls
Animal.class Guepard.class Tortue.class Zoo.class
Animal.java Guepard.java Tortue.java Zoo.java
```

► Problème :

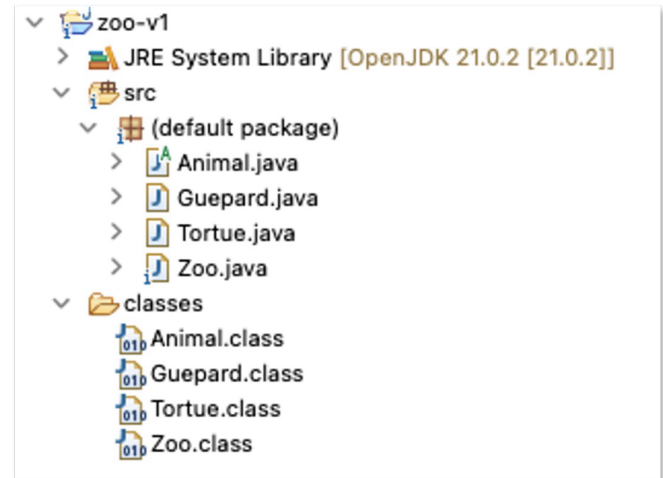
- Les fichiers source et les fichiers compilés sont mélangés dans le même répertoire



Le résultat de la compilation doit être dans un répertoire dédié nommé habituellement classes ou build/classes situé à la racine du projet

Des fichiers compilés bien rangés !

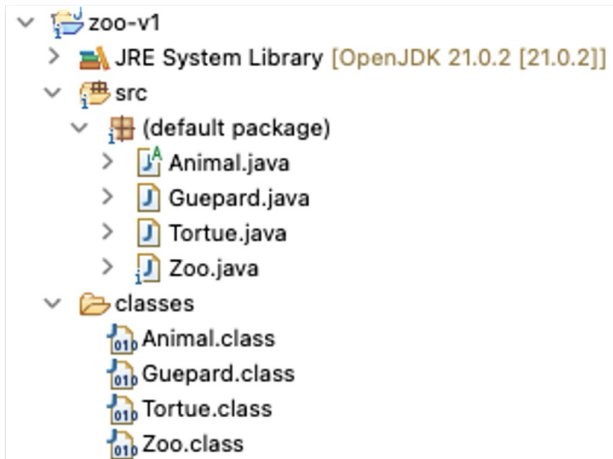
```
src$ ls
Animal.java Guepard.java Tortue.java Zoo.java
src$ javac *.java -d ../classes
src$ ls
Animal.java Guepard.java Tortue.java Zoo.java
src$ cd ..
zoo-v1$ ls
bin classes src
zoo-v1$ ls classes/
Animal.class Guepard.class Tortue.class Zoo.class
```



► Notes :

- Dans Eclipse, rafraîchir le projet (en cliquant sur le projet : F5 ou clic droit puis Refresh) pour voir le répertoire classes/
- Le répertoire bin/ à la racine du projet contient les fichiers compilés pour Eclipse qui servent pour le bon fonctionnement de l'IDE – **à ne pas utiliser !**

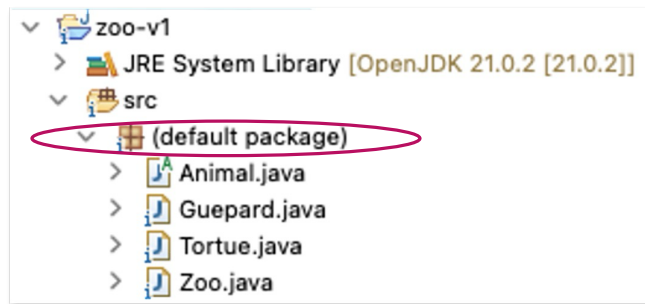
Exécuter des fichiers bien rangés



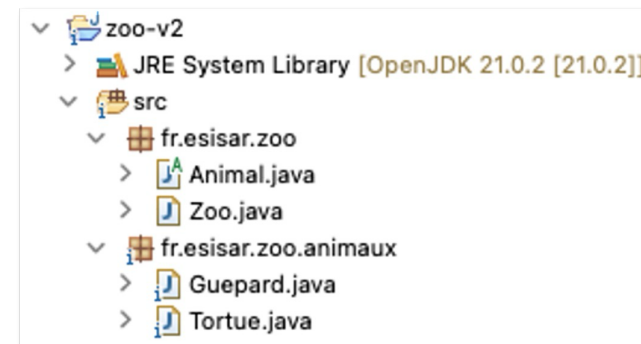
```
$ cd classes/  
$ ls  
Animal.class Guepard.class Tortue.class Zoo.class  
$ java Zoo  
Salade  
Viande  
Courir
```

Des fichiers toujours dans des packages

- Bonne pratique : les fichiers source ne doivent pas être rangés dans le « default package »



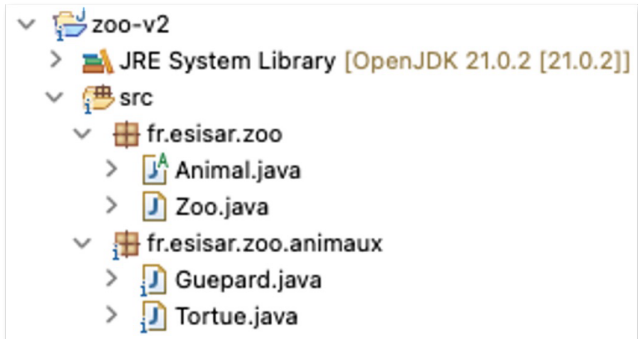
Mauvaise organisation



Bonne organisation

- Convention de nommage Java : le nom des packages doit être en minuscule :
 $^[a-z_]+(\.[a-z_][a-z0-9_]*)*\$$
 - Exemple : fr.esisar.in360

Quels sont les effets de ranger les fichiers source dans des packages ?



Arborescence :

```
$ tree -F
./
├── fr/
│   └── esisar/
│       └── zoo/
│           ├── Animal.java
│           ├── Zoo.java
│           └── animaux/
│               ├── Guepard.java
│               └── Tortue.java
5 directories, 4 files
```



```
package fr.esisar.zoo;

public abstract class Animal {

    private String cri;

    protected Animal(String cri) {
        super();
        this.cri = cri;
    }

    public abstract void manger();

    public String getCri() {
        return cri;
    }
}
```

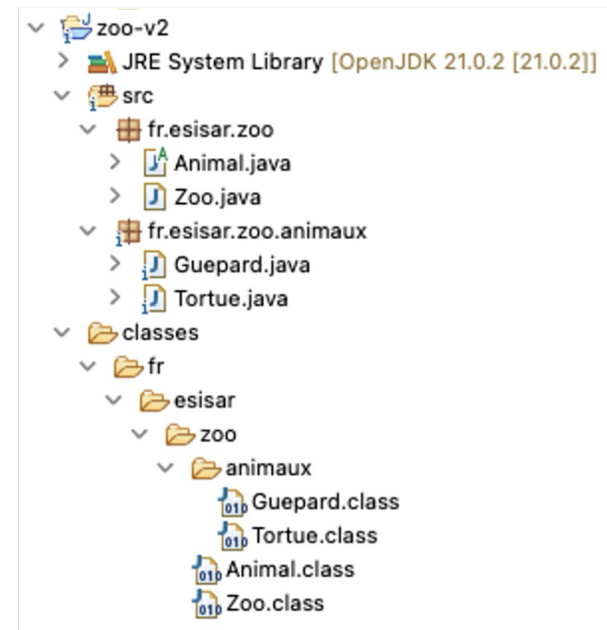
Nom pleinement qualifié de la classe est : fr.esisar.zoo.Animal

Comment compiler des fichiers rangés dans des packages ?

➡ Le résultat de la compilation doit conserver l'arborescence initiale (= conserver les sous-répertoires) qui ne doit pas être créée manuellement !



Avant compilation



Après compilation

Compilation et exécution des fichiers rangés dans des packages

- Depuis le répertoire src/

```
$ javac -d ../classes fr/esisar/zoo/*.java fr/esisar/zoo/animaux/*.java
$ tree -F ../classes
../classes/
├── fr/
│   └── esisar/
│       └── zoo/
│           ├── Animal.class
│           ├── Zoo.class
│           └── animaux/
│               ├── Guepard.class
│               └── Tortue.class
5 directories, 4 files
```

- Depuis le répertoire classes/

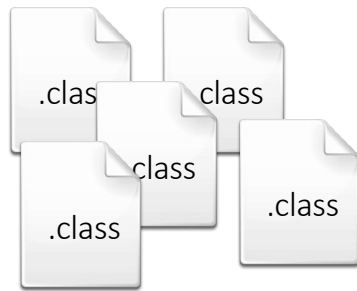
```
$ java fr.esisar.zoo.Zoo
Salade
Viande
Courir
```

Problèmes potentiels

- ▶ Les fichiers compilés ne sont pas « au bon endroit » dans le projet
- ▶ Perte de l'arborescence des packages pour les fichiers compilés
- ▶ Ajout du répertoire src/ dans l'arborescence des fichiers compilés
- ▶ Exécution de la classe principale sans son nom pleinement qualifié
 - ▶ `java Zoo` au lieu de `java fr.esisar.zoo.Zoo`
- ▶ Exécution de la classe principale en ayant mis l'extension .java ou .class
 - ▶ `java fr.esisar.zoo.Zoo.java` au lieu de `java fr.esisar.zoo.Zoo`
 - ▶ `java fr.esisar.zoo.Zoo.class` au lieu de `java fr.esisar.zoo.Zoo`

Livraison logicielle en Java

Que faut-il livrer à l'utilisateur final ?



+

Ressources :

- Bibliothèques Java
- Fichiers image
- Fichiers de configuration (XML)
- ...



Idée : Fabriquer une archive contenant tout ce qui est nécessaire

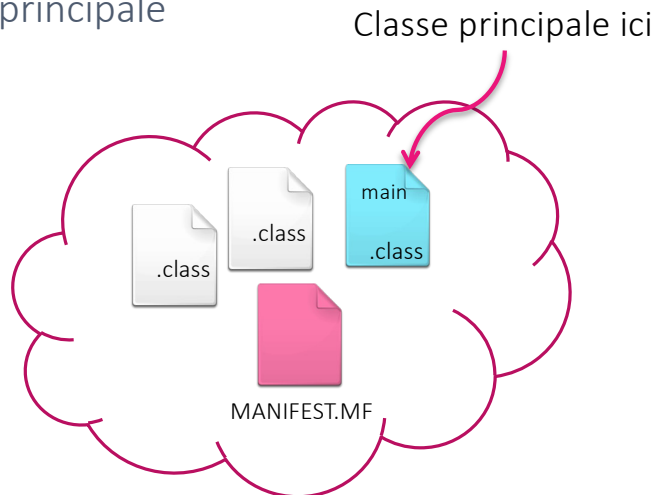
Archive Java – 1/3

- ▶ Le format de fichier **Java Archive** (JAR) permet de packager de multiples fichiers en **un unique fichier d'archive**
 - ▶ Un fichier JAR contient généralement des **fichiers compilés** et des **ressources nécessaires** à une application
- ▶ Avantages des archives Java :
 - ▶ Compression : le format JAR permet de compresser efficacement les fichiers
 - ▶ Facilite l'extension de la plate-forme Java
 - ▶ Portabilité : le mécanisme de gestion des JAR est supporté sur toutes les plates-formes Java
 - ▶ Versionnement : un JAR contient de nombreuses informations comme le vendeur, le numéro de version...
 - ▶ Sécurité : les fichiers JAR peuvent être signés

Archive Java – 2/3

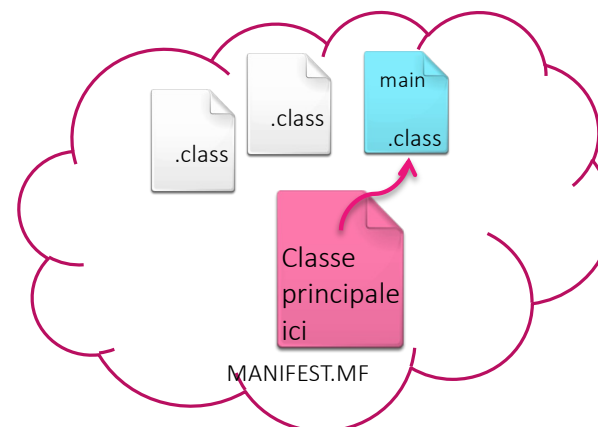
► Possibilité 1 :

- Archive non exécutable
- Nécessite à l'exécution de préciser la classe principale



► Possibilité 2 :

- Archive exécutable
- Ne nécessite pas à l'exécution de préciser la classe principale



L'arborescence des packages doit être conservée

Archive Java – 3/3

► Prérequis :

- Commande **jar** disponible dans le JDK (*Java Development Toolkit*)

► Commandes pour créer une archive (= *packager*)

► Archive non exécutable :

- `jar cvf zoo-v1.jar Zoo.class Animal.class Tortue.class Guepard.class`
- `jar cvf zoo-v1.jar -C classes/ .`



► Archive exécutable :

- `jar cvfe zoo-v1.jar Zoo Zoo.class Animal.class Tortue.class Guepard.class`
- `jar cvfe zoo-v1.jar Zoo -C classes/ .`



Exécuter un jar

- ▶ L'archive n'est pas exécutable :

```
Manifest-Version: 1.0  
Created-By: 21.0.2 (Oracle Corporation)
```

MANIFEST.MF

- ▶ `java -cp zoo-v1.jar Zoo`

- ▶ L'archive est exécutable :

```
Manifest-Version: 1.0  
Created-By: 21.0.2 (Oracle Corporation)  
Main-Class: Zoo
```

MANIFEST.MF

- ▶ `java -jar zoo-v1.jar`



Nom qualifié de la classe
Si dans un package alors :
fr.esisar.zoo.Zoo

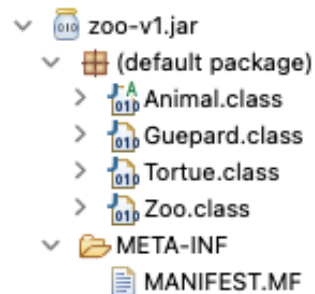
Quelle est la structure interne d'une archive JAR ?

- ▶ Il est possible de connaître le contenu d'une archive :

- ▶ En ligne de commandes :

```
$ jar tf zoo-v1.jar
META-INF/
META-INF/MANIFEST.MF
Animal.class
Guepard.class
Tortue.class
Zoo.class
```

- ▶ Depuis Eclipse (clic droit sur l'archive, Build Path > Add to Build Path)



- ▶ Ne pas oublier de retirer l'archive du classpath (clic droit, Build Path > Remove from Build Path)

Environnement Java

Quelques définitions : JVM, JDK, JRE

► JVM :

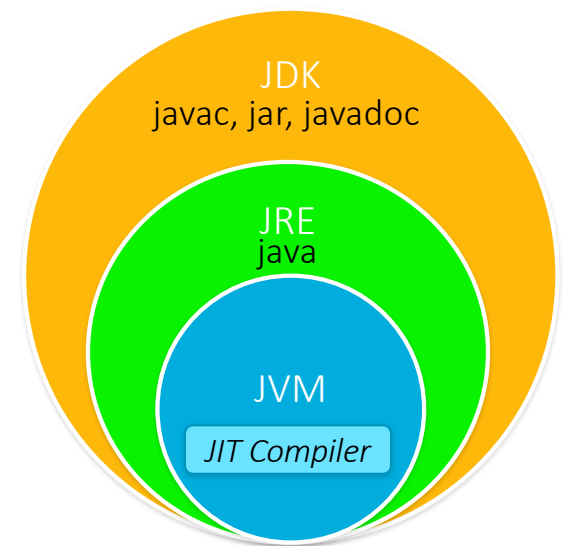
- La machine virtuelle Java (*Java Virtual Machine*) est un appareil informatique fictif qui exécute des programmes compilés sous la forme de bytecode Java.

► JRE :

- L'environnement de d'exécution (*Java Runtime Environment*) est un logiciel qui permet l'exécution des programmes écrits en langage Java (= JVM + bibliothèques)

► JDK :

- Le *Java Development Kit* est un ensemble de bibliothèques logicielles de base du langage Java ainsi que les outils avec lesquels le code Java peut être compilé, transformé en bytecode destiné à la JVM (= JRE + outils)



Que faut-il installer ?

► Attention :

- Nombreuses versions : 8, 9, 10, **11**, 12, 13, 14, 15, 16, **17**, 18, 19, 20, **21**, 22, 23
- Formats différents : **SE** (Standard Edition), ME (Micro Edition), EE (Enterprise Edition)...

► 2 possibilités :

- OpenJDK :
 - <https://jdk.java.net/archive/>
- JDK Oracle :
 - <https://www.oracle.com/java/technologies/downloads/>

21.0.2 (build 21.0.2+13)

Windows	64-bit	zip (sha256) 193M
Mac/AArch64	64-bit	tar.gz (sha256) 189M
Mac/x64	64-bit	tar.gz (sha256) 191M
Linux/AArch64	64-bit	tar.gz (sha256) 193M
Linux/x64	64-bit	tar.gz (sha256) 195M
Source		Tag jdk-21.0.2-ga



ATTENTION :

Par défaut, les machines personnelles ont toutes un JRE mais pas de JDK
Il faut absolument installer un JDK !
Risque de conflits entre le JRE par défaut et votre installation de JDK !

Configuration machine

- ▶ Prérequis :
 - ▶ Installer un JDK
- ▶ Configuration des variables d'environnement :
 - ▶ JAVA_HOME : pointe sur le répertoire du jdk
 - ▶ PATH : pointe sur le répertoire /bin du jdk
 - ▶ Linux : `$PATH=$JAVA_HOME/bin:$PATH`
 - ▶ Windows : `%PATH%=%JAVA_HOME%\bin;%PATH%`

- ▶ Vérification de la configuration

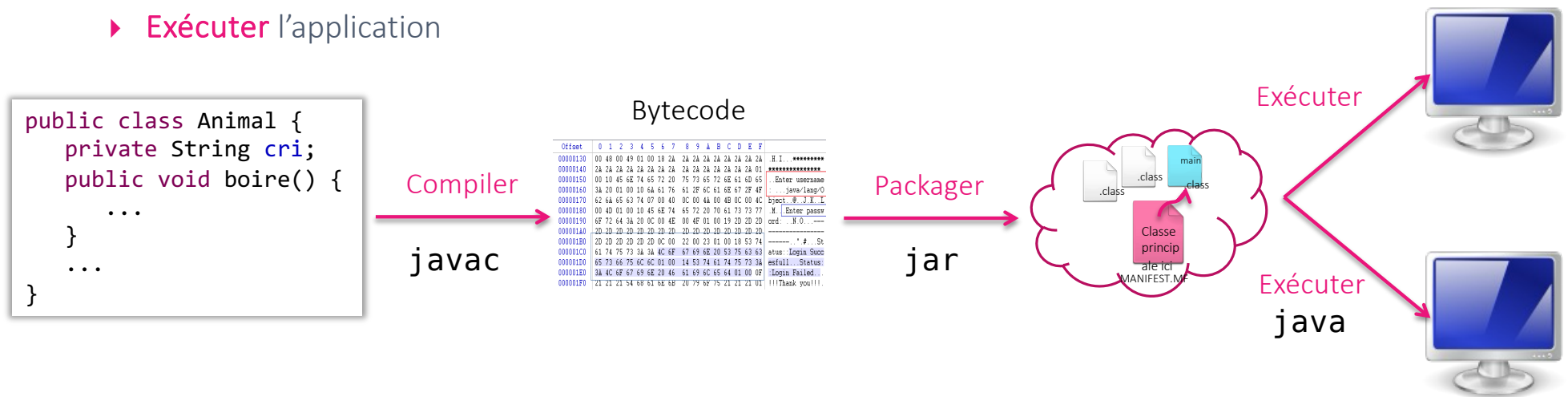
ET

- ▶ `java -version`
- ▶ `javac -version`

 } Doivent donner **EXACTEMENT** le même numéro de version !

Synthèse

- Pour réaliser une application Java, il faut :
 - **Configurer** correctement sa machine = avoir un JDK et JRE dans la même version
 - **Configurer** son IDE avec la même version que sa machine (les IDE ont par défaut leur JDK et JRE)
 - **Compiler** le code source
 - **Packager** (= créer l'archive) de l'application
 - **Exécuter** l'application



Automatisation des tâches

Apache Ant

Build Process en C

► Exemple : Makefile

```
CC = gcc
CFLAGS = -Wall -ansi -pedantic

all: main

main: acquisition.o stockage.o traitement.o main.o
    $(CC) $(CFLAGS) -o $@ $^

.o: %.c %.h
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm *.o main data*
```



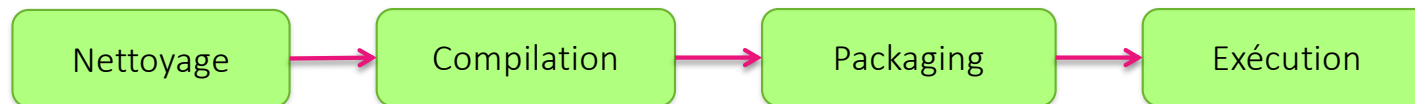
En Java ?

Build Process en Java

► Quelles sont les tâches récurrentes à automatiser ?

- **Compilation**
- **Packaging**
- Déploiement
- **Exécution**
- Tests
- Documentation
- ...

► Dans quel ordre ?



Deux outils d'automatisation des tâches, deux philosophies

► Apache Ant

- <http://ant.apache.org/>
- Configuration à la charge du développeur pour faciliter la portabilité :
 - Ensemble de fonctionnalités qui ont le même comportement sur tous les systèmes
- Fichier build.xml



► Apache Maven

- <https://maven.apache.org/>
- Convention plutôt que configuration :
 - Maven impose une arborescence et un nommage des fichiers du projet
- Fichier pom.xml

maven

Apache Ant¹



- ▶ Objectif :
 - ▶ **Automatiser les opérations répétitives** du développement logiciel telles que la compilation, la génération de documentation ou l'archivage au format Jar.
- ▶ Apache Ant est développé en Java
- ▶ Principalement utilisé pour des projets Java mais peut être utilisé pour tout autre type d'automatisation dans n'importe quel langage

¹ Acronyme de *Another Neat Tool* (« Un autre outil chouette »)

Fonctionnement de Apache Ant

- ▶ Description de l'ensemble des tâches qui doivent être effectuées de manière indépendante de la plate-forme d'exécution
 - ▶ Les tâches sont décrites en XML dans un fichier de configuration build.xml
- ▶ Intégration de Ant dans un projet Java
 - ▶ Ajout du fichier **build.xml** à la racine du projet



Rapide introduction au XML – 1/3

- ▶ XML (*eXtensible Markup Language*) est un **langage de balisage** conçu pour stocker, structurer des données de manière lisible pour l'homme et la machine
 - ▶ Recommandation du W3C (depuis 1998)
- ▶ Structure de base :

Commentaires

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Fichier XML représentant une bibliothèque de livres -->
```

Élément racine

```
<bibliotheque>
```

Balise ouvrante

```
  <livre id="L101" categorie="fantasy">
```

```
    <titre>Harry Potter à l'école des sorciers</titre>
```

```
    <auteur>J.K. Rowling</auteur>
```

```
    <annee>1997</annee>
```

Balise auto-fermante

```
    <couverture src="hp1.jpg"/>
```

```
  </livre>
```

```
</bibliotheque>
```

Déclaration XML

Balise fermante

Attribut Valeur de l'attribut

Rapide introduction au XML – 2/3

► Bonnes pratiques :

- Un unique élément racine
- Chaque balise ouverte doit être fermée
 - Avec une balise ouvrante et une balise fermante : `<annee>1997</annee>`
 - Avec une balise auto-fermante : `<couverture src="hp1.jpg" />`



- Règle d'imbrication des balises XML :
 - On referme toujours la dernière balise ouverte
 - Pas obligatoire mais plus lisible : indentation après chaque balise ouverte
- Une balise peut avoir des attributs. Chaque attribut est unique. Les attributs ne sont pas ordonnés. La valeur d'un attribut doit être entre guillemets.

Rapide introduction au XML – 3/3

► Principe général :

- ▶ Les utilisateurs peuvent définir leurs propres balises
- ▶ Les balises indiquent la signification des sections : un objet balisé s'auto-décrit
- ▶ Il est possible d'imposer une grammaire spécifique (DTD ou XML Schema) ce qui impose le vocabulaire et l'organisation des balises et des attributs

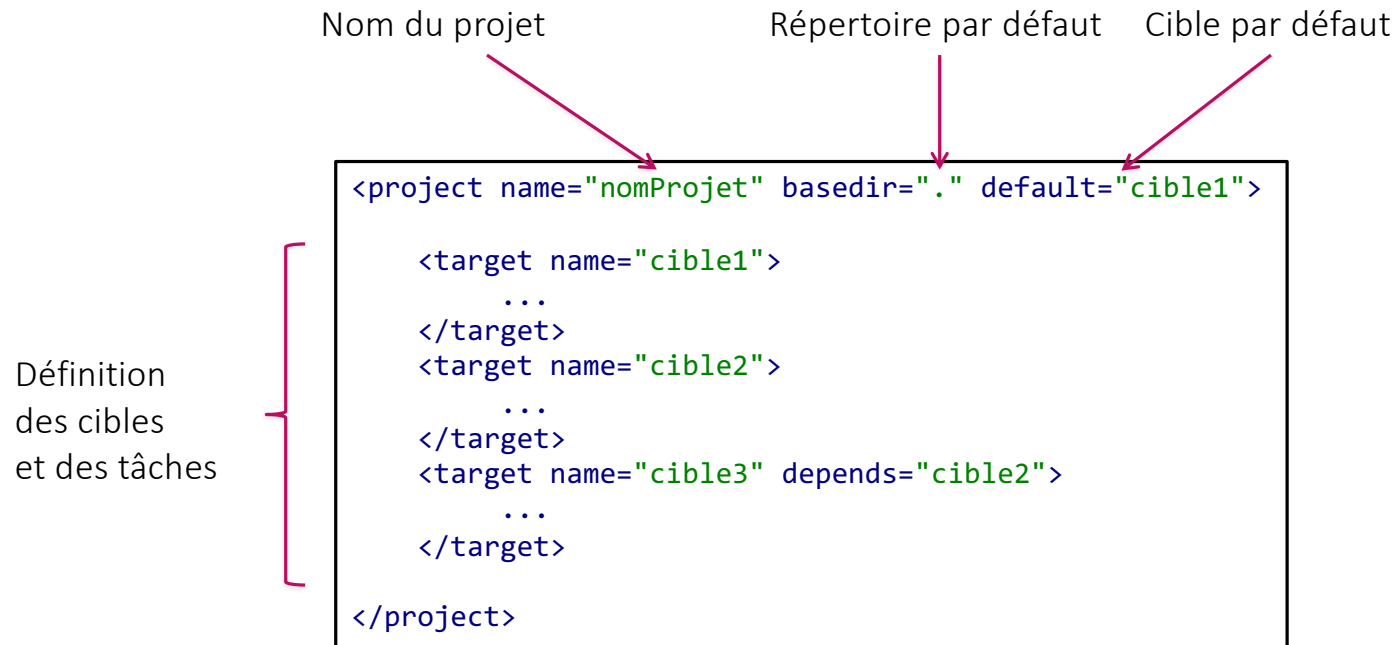
► Intérêts du XML :

- ▶ Lisible par un humain : syntaxe claire et hiérarchique
- ▶ Standard universel
- ▶ Auto-descriptif et extensible

► Limites du XML :

- ▶ Très verbeux : beaucoup de texte pour une petite quantité d'information (versus JSON, YAML)
- ▶ Nécessite de la rigueur pour l'écrire manuellement

Structure d'un fichier build.xml



Exemples de tâches simples

► Création d'un répertoire

```
<mkdir dir="rep"/>
```

► Suppression d'un fichier ou d'un répertoire

```
<delete file="file.jar"/>
```

```
<delete dir="rep"/>
```

► Copie d'un fichier ou d'un répertoire

```
<copy file="file.txt" tofile="myCopy.txt"/>
```

```
<copy file="file.txt" todir="../some/other/rep"/>
```

```
<copy todir="../some/other/dest_dir">  
  <fileset dir="src_dir" />  
</copy>
```

*Copie le répertoire src_dir
dans le répertoire dest_dir*

Exemples de tâches spécifiques à Java – 1/3

► Compilation : javac

Attribut	Rôle
srcdir	précise le répertoire racine de l'arborescence contenant les sources
destdir	précise le répertoire où les résultats des compilations seront stockés
classpath	classpath pour l'exécution. Il est aussi possible d'utiliser un tag fils <classpath> pour le spécifier

► Exemple : `<javac srcdir="src_dir" destdir="dest_dir" includeantruntime="false"/>`

Exemples de tâches spécifiques à Java – 2/3

► Exécution : java

Attribut	Rôle
classname	nom pleinement qualifié de la classe à exécuter
jar	nom du fichier de l'application à exécuter
classpath	classpath pour l'exécution. Il est aussi possible d'utiliser un tag fils <classpath> pour le spécifier
fork	lancer l'exécution dans une JVM dédiée au lieu de celle où s'exécute Ant

► Exemples :

```
<java classname="test.Main" fork="true"/>
```

```
<java classname="test.Main" fork="true">  
  <arg value="-h"/>  
  <classpath>  
    <pathelement location="dist/test.jar"/>  
  </classpath>  
</java>
```

Exemples de tâches spécifiques à Java – 3/3

► Packaging : jar

Attribut	Rôle
destfile	nom du fichier .jar à créer
basedir	précise de répertoire qui contient les éléments à ajouter dans l'archive
manifest	précise le fichier manifest qui sera utilisé dans l'archive

► Exemples :

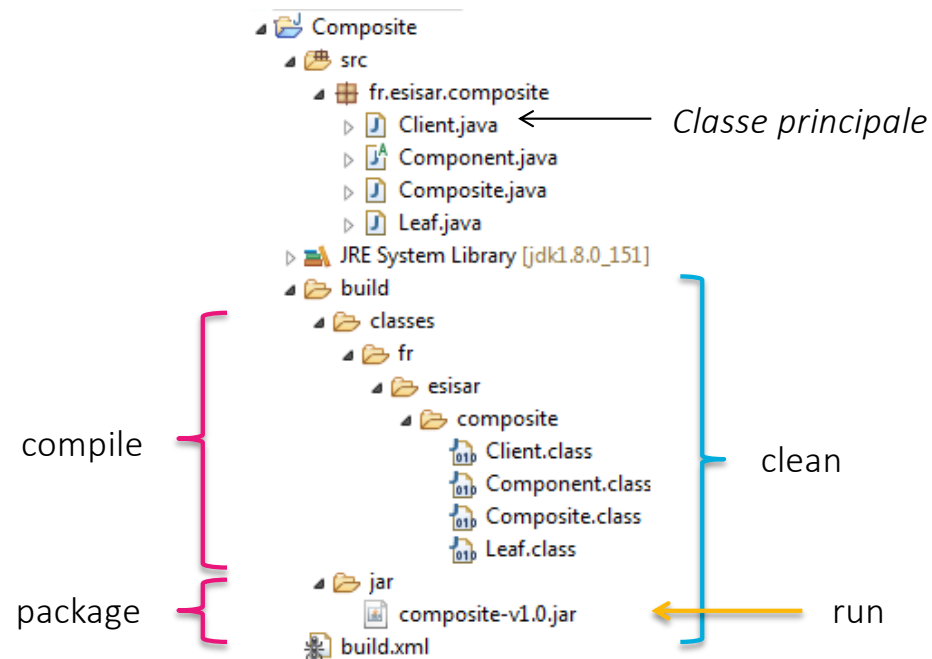
► Jar non exécutable :

```
<jar destfile="monJar.jar" basedir="classes"/>
```

► Jar exécutable :

```
<jar destfile="monJar.jar" basedir="classes">  
  <manifest>  
    <attribute name="Main-Class" value="fr.esisar.MaClasse"/>  
  </manifest>  
</jar>
```


Exercice : écrire le fichier build.xml



Pour aller plus loin...

- Il est possible de définir des propriétés pour paramétrer le fichier build.xml. Ces propriétés doivent être définies après la balise <project> et avant la première balise <target>

```
<property name="var" value="value"/>
```

- Utilisation d'une propriété :

```
${var}
```

- Il existe des propriétés prédéfinies dans Ant :

basedir	chemin absolu du répertoire de travail (cette valeur est précisée dans l'attribut basedir du tag project)
ant.file	chemin absolu du fichier build en cours de traitement
ant.java.version	version de la JVM qui exécute ant
ant.project.name	nom du projet en cours d'utilisation

- ▶ Objectif :
 - ▶ **Automatiser les opérations répétitives** du développement en général, plus particulièrement des projets Java EE, suivant les principes de l'intégration continue

- ▶ Principe :
 - ▶ Convention plutôt que configuration
 - ▶ Pour la structure du projet
 - ▶ Pour le cycle de vie (compilation, test, packaging, installation, déploiement)

- ▶ Basé sur un fichier pom.xml
- ▶ Utilise un dépôt d'archives (distant et local)



Cours de *Génie Logiciel et BD* de l'année prochaine

Gradle

► Objectif :

- Automatisation des tâches pour construire des projets en Java, Scala, Groovy, Android, Kotlin, Swift, C++...

► Principe :

- Utilise les conventions à la manière de Maven
- Utilise la flexibilité de Ant pour décrire les tâches de construction

Conclusion

Bilan

► Compétences acquises :

- Compréhension de la chaîne de production pour les applications Java
- Maîtrise des commandes de la chaîne de production en ligne de commandes
- Maîtrise de l'automatisation de la chaîne de production avec Apache Ant

